GO-LIVE CHECKLIST · 31 JULY 2026

What it takes to make Alsaiti Voice 100% live

Every remaining task, who can do it, and the DNS records for production. The code is written and tested; almost everything left needs an account login rather than an engineer.

THE SITUATION IN ONE PARAGRAPH

The website is live and the contact-form fix is written, tested and committed — but the version running in production is still the old one. It shows “Sent” and saves the enquiry, then tells nobody. Every other item in this document is downstream of one deployment command that takes about ten minutes.

Where things stand

AREA	STATE	WHY
Contact form (front end)	Live	Waits for the server, keeps the visitor’s text on failure, quotes a reference
Contact form (backend)	Old build	Written & tested, not deployed. No validation, no de-duplication, no email
Lead alert email	404	lead-notify not deployed
Health endpoint	404	Not deployed
Migrations 0005–0007	Not applied	The leads feature cannot work until these land
Leads in Supabase	Built	A real account reads and writes the database; the demo stays browser-local
Retention job	Needs pg_cron	Written. The published privacy policy promises it
Uptime alerting	Needs a monitor	Endpoint exists; nothing watches it
Email delivery	Needs DNS	Until verified, Resend only mails your own address
Tenant isolation proof	Needs 2 accounts	Enforced in the schema; the test is written and waiting
HubSpot / Telnyx	Needs accounts	Code deployed, no credentials
Live phone answering	Worker built	Logic tested. Needs LiveKit, a funded Telnyx number and a host

Part 1 — The critical path

In order, because each unblocks the next. Under an hour of work in total, plus waiting for DNS to propagate.

1. Deploy the database and the functions

The database goes first: the migrations create the columns the new contact form writes into. Deploy the function first and every submission fails.

```
npm i -g supabase
supabase login
supabase link --project-ref jnxvwdcvnwigowafdxvl
supabase db push
supabase functions deploy contact-submit lead-notify health
```

Then confirm it actually took — this needs no credentials and checks every endpoint from the outside:

```
node tests/verify-deploy.js
```

Run against production today it reports five failures, including that *contact-submit accepts an invalid email address* — which is how you can tell it is still the old build. When it prints "Deploy verified", move on.

2. Set the two missing email secrets

The Resend API key is already in place. On its own it sends nothing — the function also needs a From address and a destination, or it returns `no_recipient` and stays silent.

```
supabase secrets set \
ALERT_FROM_EMAIL="Alsaiti Growth <alerts@send.alsaitigrowth.com>" \
LEAD_NOTIFICATION_TO="contact@alsaitigrowth.com"
```

3. Verify the sending domain

Records are in Part 2. Until the domain shows **Verified**, Resend delivers only to your own account address — alerts to anyone else fail silently, which is exactly the failure this whole project set out to remove.

4. Enable pg_cron and confirm the job exists

Supabase → Database → Extensions → enable `pg_cron`, then re-run migration 0007. The migration does not fail when the extension is off — it prints a notice and moves on — so check for yourself:

```
select jobname, schedule from cron.job where jobname = 'alsaiti_retention_sweep';
```

No row means nothing is deleting old data, and the privacy policy published on the website is a promise the system does not keep.

5. Delete the junk test row

Mine, from probing whether the endpoint validated email addresses. It did not.

```
delete from contact_submissions where email = 'notanemail';
```

6. Point an uptime monitor at the health endpoint

Better Stack, Cronitor, UptimeRobot — any of them. Five-minute interval, alert on any non-2xx response. The endpoint returns **503** when degraded, so this needs no other configuration.

```
https://jnxvwdcvnwigowafdxvl.supabase.co/functions/v1/health
```

Without this, a broken lead pipeline is visible in the dashboard but nothing tells anybody.

7. Prove tenant isolation

Create a second account in Supabase → Authentication → Users, then run the test. It creates a probe lead in one workspace and tries every route the other account might use to reach it — list, read by id, filter by workspace, update, insert, delete — across five tables, then cleans up after itself.

```
SUPABASE_URL=https://jnxvwdcvnwigowafdxvl.supabase.co \
SUPABASE_ANON_KEY=sb_publishable_... \
A_EMAIL=owner-a@example.com A_PASS=... \
B_EMAIL=owner-b@example.com B_PASS=... \
node tests/isolation-live.js
```

8. Migrate off the legacy Supabase keys

Order matters and the wrong order takes the whole backend down. Set the new keys, smoke-test, and only then disable the old ones. The code falls back, so a bad step is undone by unsetting a secret.

```
# 1. create sb_secret_... in Project Settings -> API, then:
supabase secrets set SUPABASE_SECRET_KEY=... SUPABASE_PUBLISHABLE_KEY=...
# 2. smoke-test: submit the contact form, open Integrations, play a TTS line
# 3. only now: disable legacy keys in the dashboard
```

9. Rotate the demo password

It sits in a public static file, so rotating does not hide it. What matters is that the demo account cannot reach production data or provider settings. Treat the credential as public by design and enforce the isolation instead.

10. Have a solicitor read the privacy policy and terms

They were written to match what the code actually does, which is accuracy, not legal advice. They now carry retention commitments and a liability cap.

Part 2 — DNS records for production

Add the domain at **resend.com/domains** and it generates four records. Choose the **eu-west-1** region so mail sits in the same place as the database.

TYPE	NAME	VALUE	PURPOSE
MX	`send`	`feedback-smtp.eu-west-1.amazonses.com priority 10`	Bounce and complaint handling
TXT	`send`	`v=spf1 include:amazonses.com ~all`	SPF — authorises the sender
TXT	`resend._domainkey`	**`p=MIGfMA0GCSqGSIb3DQEB…`** `copy from your dashboard`	DKIM — cryptographically signs your mail
TXT	`_dmarc`	`v=DMARC1; p=none;`	DMARC — optional, worth having

THE DKIM VALUE IS NOT PRINTED HERE, ON PURPOSE

It is a public key generated uniquely for your domain the moment you add it in Resend. Nobody can supply it without opening your dashboard — so treat anyone who hands you a “complete” set of four records as guessing.
Copy it from resend.com/domains exactly. It runs to roughly 400 characters and must not be wrapped onto a second line.

Two things that waste an afternoon

- If your DNS host appends the domain automatically, enter `send` and not `send.alsaitigrowth.com` — otherwise you end up with `send.alsaitigrowth.com.alsaitigrowth.com`.
- Propagation runs from a few minutes to a few hours. A failed verify is not proof the records are wrong — wait, then check again.

Part 3 — Beyond the website

These are the difference between a working website and the full product. The site is honest about all three today — each is labelled *Needs setup* or *Infrastructure required* rather than pretending.

HubSpot

OAuth, encrypted token storage, metadata loading and lead sync are written and deployed. Create a developer app at developers.hubspot.com and store its two credentials as Supabase secrets. **Do not send them to anyone, including a developer** — enter them in the dashboard directly.

ITEM	VALUE
Redirect URL (must match exactly)	`https://jnxvwdcvnwigowafdxvl.supabase.co/functions/v1/crm-callback`
Scopes	crm.objects.contacts.read / .write, crm.objects.deals.read / .write, crm.schemas.contacts.read, crm.schemas.deals.read, oauth
Secrets to set	`HUBSPOT_CLIENT_ID, HUBSPOT_CLIENT_SECRET`

Telnyx

Number search, ordering and signed webhook verification are written and deployed. Needs a verified, funded account before anything can answer. Set `TELNYX_API_KEY` and `TELNYX_PUBLIC_KEY` — webhook verification fails without the second one. Webhook URL:

```
https://jnxvwdcvnwigowafdxvl.supabase.co/functions/v1/telnyx-webhook
```

Live phone answering

The worker is now **built**, in `voice-worker/`. Everything a business would hold it to — extraction, qualification, transfer rules, and the guarantee that one call creates exactly one lead — is covered by 40 tests that run with no network and no credentials.

One file, the LiveKit binding, has never been run, because that needs a real deployment, a real SIP trunk and a real phone call. It says so at the top of the file. It is deliberately thin so the rules around it stay stable when the SDK moves.

Still required from you: a LiveKit deployment and its three credentials; a funded Telnyx number with an inbound trunk pointed at LiveKit; somewhere always-on to run the worker, and approval for the monthly cost; the phone number a human answers for urgent calls; and a decision on call recording and the consent wording your jurisdiction requires.

> **DO THIS BEFORE THE FIRST REAL CALL, NOT AFTER**
> Test CALL-09 asks what happens when the worker itself is down. Configure carrier fallback to voicemail or call forwarding **before** going live. Nothing may be labelled “Live” until all ten call tests pass against a real phone.

Part 4 — What “100%” actually means

Worth being precise, because the two answers are far apart.

MILESTONE	WHAT IT TAKES	REALISTIC
The website fully working	Part 1, items 1–10. Nearly all of it is dashboard logins rather than engineering.	**A few days**, mostly spent waiting on DNS

MILESTONE	WHAT IT TAKES	REALISTIC
The full product working	All of the above, plus HubSpot proven, Telnyx funded and proven, and the voice worker deployed and passing all ten call tests against a real phone.	**Weeks** — gated on provider accounts and monthly hosting spend, not on code

Where the code stands today

Web acceptance suites	**13 of 13 green** — 350+ individual checks
Voice worker tests	**40 of 40 green**
Edge Functions	**14 of 14 type-check clean**
Native app	**21 of 21 render checks**, static audit clean
Secret scan	**Clean** — working tree and full git history
Continuous integration	Runs all of the above on every push

Never send an API key, access token, client secret, password or recovery code through chat, email or a screenshot — including to a developer. Enter them directly into the dashboard or secret manager named above.